LeetCode Practice

5 Hard: Math

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

5
HARD

Patterns (in order)

1. Math (5)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

5 questions | tap a title to jump to it

Hard questions (5) Go >

Math (5) Go >

H 1. Text Justification LC 68

H 2. Max Points on a Line LC 149

H 3. Integer to English Words LC 273

H 4. Perfect Rectangle LC 391

H 5. Guess the Word LC 843

Hard questions (5)

HARD | TOPIC 1 OF 1

Math

5 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Text Justification LC 68
2. Max Points on a Line LC 149
3. Integer to English Words LC 273
4. Perfect Rectangle LC 391
5. Guess the Word LC 843

HARD | Math | LeetCode #68 | 1 of 5 | 1/5

Text Justification

QUESTION

Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly maxWidth characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

CONSTRAINTS

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed maxWidth.
- The input array words contains at least one word.

Example 1

Input	`words = ["This", "is", "an", "example", "of", "text", "justification."], maxWidth = 16`
Output	`[ "This is an", "example of text", "justification. " ]`

Example 2

Input	`words = ["What","must","be","acknowledgment","shall","be"], maxWidth = 16`
Output	`[ "What must be", "acknowledgment ", "shall be " ]`
Explanation	*Note that the last line is "shall be " instead of "shall be", because the last line must be left-justified instead of fully-justified. Note that the second line is also left-justified because it contains only one word.*

QUESTION

Example 3

Input	`words = ["Science","is","what","we","understand","well","enough","to","explain","to","a","computer.","Art","is","everything","else","we","do"], maxWidth = 20`
Output	`[ "Science is what we", "understand well", "enough to explain to", "a computer. Art is", "everything else we", "do " ]`

- 1 <= words.length <= 300
- 1 <= words[i].length <= 20
- words[i] consists of only English letters and symbols.
- 1 <= maxWidth <= 100
- words[i].length <= maxWidth

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Arrange words into lines of exactly maxWidth chars. Spaces distributed as evenly as possible.

Extra spaces go to the LEFT. Last line: left-justified with single spaces. Right?"

"A few quick questions: Each word fits in maxWidth? Yes per constraints.

At least one word? Yes."

"Let me walk: words=['This','is','an','example'], maxWidth=16.

Line 1: 'This is an' (total=16) ok. Last: left-justified.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
i = 0
while i < len(words):
    j = i + 1
    while joining words[i..j] with single spaces
        still fits in width:
        j = j + 1
    spread the spare spaces over the gaps
        (left-justify the last line)
    i = j
```

TIME	SPACE
O(total characters * words per line)	O(width)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Re-joins the candidate line after every added word, so the same characters are scanned again and again. Keeping a running length (sum of word lengths plus minimum gaps) makes the whole job O(total characters).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Greedy line filling + space distribution — O(n * L). Walk-through:"*

Greedily fill each line: pack as many words as fit (with single spaces between).

For each filled line:

If last line OR only one word: left-justify (spaces on right, single space between words).

Else: distribute remaining spaces evenly:

`gaps = len(words) - 1. extra = maxWidth - sum(word lengths).`

`spaces_per_gap = extra // gaps. leftover = extra % gaps.`

First 'leftover' gaps get one extra space.

WHY leftover to left gaps: problem specifies extra spaces go to left gaps first.

HOW to fill a line: maintain curr_width, keep adding words until next word would overflow.

Time: O(n * maxWidth). Space: O(maxWidth) per line.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _68_TextJustification:
    def fullJustify(self, words, maxWidth):
        result = []
        i = 0
        while i < len(words):
            line = [words[i]]; length = len(words[i]); i +
= 1
            while i < len(words) and length + 1 + len(word
s[i]) <= maxWidth:
                length += 1 + len(words[i]); line.append(w
ords[i]); i += 1
            if i == len(words) or len(line) == 1:
                result.append(' '.join(line).ljust(maxWidt
h))
            else:
                spaces = maxWidth - sum(len(w) for w in li
ne)
                gaps = len(line) - 1
                base, extra = divmod(spaces, gaps)
                row = line[0]
                for j in range(1, len(line)):
                    row += ' ' * (base + (1 if j <= extra
else 0)) + line[j]
                result.append(row)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the derived formula and confirm it on the example numbers."

words=['This','is','an','example'], maxWidth=16:

line=['This','is','an'], length=10. spaces=6, gaps=2. base=3,extra=0.

row='This is an'. len=16 ok

Last line='example '.ljust(16) ok

Single word line: 'word'.ljust(16) -> padded with spaces ok

Base + (1 if j <= extra else 0) distributes extra spaces to left gaps

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*maxWidth)	**O(n)**
O(n) lines, each O(maxWidth) to build	output

Follow-up: minimize raggedness (aesthetic) — DP approach balances line lengths.

Follow-up: proportional fonts — use pixel widths instead of character counts.

HARD | Math | LeetCode #149 | 2 of 5 | 2/5

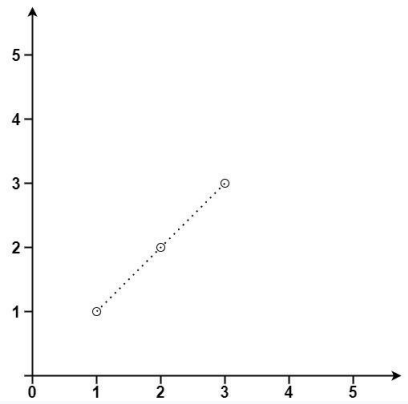
Max Points on a Line

QUESTION

Given an array of points where points[i] = [x_i, y_i] represents a point on the X-Y plane, return the maximum number of points that lie on the same straight line.

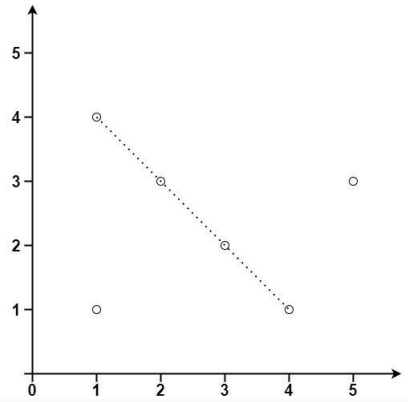
Example 1

Input	`points = [[1,1],[2,2],[3,3]]`
Output	3



Example 2

Input	`points = [[1,1],[3,2],[5,3],[4,1],[2,3],[1,4]]`
Output	4



CONSTRAINTS

- 1 <= points.length <= 300
- points[i].length == 2
- 10^4 <= x_i, y_i <= 10^4
- All the points are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the maximum number of points that lie on the same straight line. Right?"

"A few quick questions:

Can points be duplicate? Yes — they're all on the same 'line' trivially. Handle with care."

"Let me walk: points=[[1,1],[2,2],[3,3]] -> 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (i, j) in pairs(points):
    count the points k that lie on the line
        through them (cross product is zero)
    best = max(best, count)
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

n^2 lines, each tested against all n points. For each point, count equal slopes to every other point in a hash map (reduced fraction as the key): O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Hash slopes from each anchor point — O(n²). Walk-through:"

For each point i as anchor:

Build slope map: slope -> count of points on that line through i.

For each j != i:

Compute slope as reduced fraction (dy/gcd, dx/gcd) to avoid float precision issues.

Handle verticals (dx==0) and horizontals (dy==0) as special keys.

slope_count[slope]++.

`best = max(best, max(slope_count.values()) + 1) (+1 for the anchor itself).`

WHY rational fractions: floating-point slopes cause precision errors. Store as (dy//g, dx//g)

where g = gcd(|dy|, |dx|) — canonical form for the slope.

WHY sign convention: normalize so denominator > 0 (or handle sign consistently).

Time: O(n²). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import defaultdict
from math import gcd
class _149_MaxPointsOnALine:
    def maxPoints(self, points):
        n = len(points)
        if n <= 2:
            return n
        best = 2
        for i in range(n):
            slope_count = defaultdict(int)
            duplicates = 0
            for j in range(i + 1, n):
                dx = points[j][0] - points[i][0]
                dy = points[j][1] - points[i][1]
                if dx == 0 and dy == 0:
                    duplicates += 1
                    continue
                g = gcd(abs(dx), abs(dy))
                dx //= g; dy //= g
                if dx < 0 or (dx == 0 and dy < 0):
                    dx, dy = -dx, -dy
                slope_count[(dy, dx)] += 1
            local_best = max(slope_count.values(), default
=0)
            best = max(best, local_best + 1 + duplicates)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace maxPoints([[1,1],[2,2],[3,3]])."

For each point as anchor, count slopes to all other points.

Anchor (1,1): slope to (2,2)=(1,1) (dy=1,dx=1->gcd=1->slope=(1,1)). slope to (3,3)=(2,2)->gcd=2->(1,1). Both same slope! max=2.

Total including anchor: 2+1=3 ok

WHY use fraction not float: float precision issues. Represent slope as (dy//g, dx//g) normalized.

Handle dx=0 (vertical) and dy=0 (horizontal) as special slope keys.

Edge: all points on same line -> slope all same -> max=n-1+1=n ok

WHY anchor-based: for each anchor, slopes to others -> same-slope = same line through anchor.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	$O(n)$ for slope map

Follow-up: if points are in 3D, extend to plane equation fitting.

Follow-up: if we want the actual line, track which slope had max count.

HARD | Math | LeetCode #273 | 3 of 5 | 3/5

Integer to English Words

QUESTION

Convert a non-negative integer num to its English words representation.

Example 1

Input	num = 123
Output	"One Hundred Twenty Three"

Example 2

Input	num = 12345
Output	"Twelve Thousand Three Hundred Forty Five"

Example 3

Input	num = 1234567
Output	"One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven"

CONSTRAINTS

- $0 <= num <= 2^{31} - 1$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Convert a non-negative integer to its English words representation. Right?"

"A few quick questions:

Range is [0, 2^31-1]. num=0 -> 'Zero'? Yes."

"Let me walk: num=1234567 -> 'One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven' ok

PHASE 2 BRUTE FORCE (also optimal)

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
build a table of English words for every number 0 to
    999
split the number into groups of three digits and
    attach Thousand, Million, Billion
```

TIME	SPACE
O(1)	O(1000)

NOTE

Not slow: the input has at most 10 digits. Precomputing a thousand strings wastes memory; small tables for ones, teens and tens, applied per three-digit group, do the same work.

PHASE 2 BRUTE FORCE (also optimal)

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Recursive chunking by magnitude — O(1) since num <= 2^{31}. Walk-through:"

Handle billions, millions, thousands, hundreds recursively:

say(num):

If num == 0: return ''.

If num < 20: return ones[num].

If num < 100: return tens[num//10] + (ones[num%10] if num%10 else '').

If num < 1000: return ones[num//100] + ' Hundred' + say(num%100).

For each chunk (billion=10, million=10, thousand=10^3, else):

If num >= chunk: return say(num//chunk) + ' ' + label + say(num%chunk).

WHY recursive chunking: English naming is naturally hierarchical — billions contain millions

which contain thousands which contain hundreds which contain small numbers.

Time: O(1). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _273_IntegerToEnglishWords:
    def numberToWords(self, num):
        if num == 0:
            return 'Zero'
        ONES = ['','One','Two','Three','Four','Five','Six'
,'Seven','Eight','Nine',
                'Ten','Eleven','Twelve','Thirteen','Fourte
en','Fifteen','Sixteen',
                'Seventeen','Eighteen','Nineteen']
        TENS = ['','','Twenty','Thirty','Forty','Fifty','S
ixty','Seventy','Eighty','Ninety']
        def helper(n):
            if n == 0: return ''
            if n < 20: return ONES[n] + ' '
            if n < 100: return TENS[n // 10] + ' ' + helpe
r(n % 10)
            return ONES[n // 100] + ' Hundred ' + helper(n
 % 100)
        result = ''
        for value, name in [(10**9,'Billion'),(10**6,'Mill
ion'),(10**3,'Thousand'),(1,'')]:
            if num >= value:
                result += helper(num // value) + (name + '
 ' if name else '')
                num %= value
        return ' '.join(result.split())
```

PHASE 5 TEST & DEBUG

"Let me trace numberToWords(1234567)."

1234567 = 1 Million + 234 Thousand + 567

1 -> 'One Million'

234 -> helper(234): 2 Hundred + 34 = 'Two Hundred Thirty Four' -> 'Two Hundred Thirty Four Thousand'

567 -> helper(567): 5 Hundred + 67 = 'Five Hundred Sixty Seven'

Join: 'One Million Two Hundred Thirty Four Thousand Five Hundred Sixty Seven' ok

Recursive helper for <=999: handles hundreds, tens (10-19 special), ones separately.

Edge: n=0 -> 'Zero'; n=1000000000 -> 'One Billion'.

WHY chunk by 1000: English groups thousands naturally.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1) at most 13 groups	**O(1)**

Follow-up: English Words to Integer — reverse parsing; split on thousand/million/billion.

Follow-up: Roman to Integer (#13) — similar digit-group decomposition.

HARD | Math | LeetCode #391 | 4 of 5 | 4/5

Perfect Rectangle

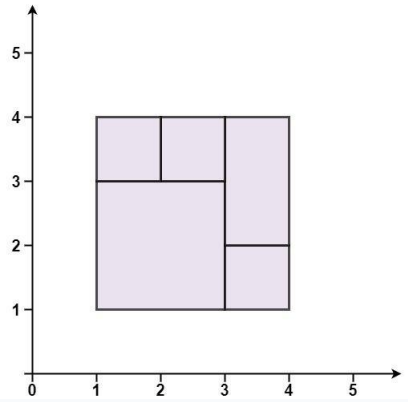
QUESTION

Given an array rectangles where rectangles[i] = [x_i, y_i, a_i, b_i] represents an axis-aligned rectangle. The bottom-left point of the rectangle is (x_i, y_i) and the top-right point of it is (a_i, b_i).

Return true if all the rectangles together form an exact cover of a rectangular region.

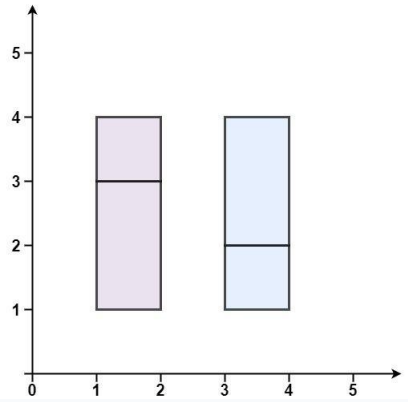
Example 1

Input	`rectangles = [[1,1,3,3],[3,1,4,2],[3,2,4,4],[1,3,2,4],[2,3,3,4]]`
Output	`true`
Explanation	*All 5 rectangles together form an exact cover of a rectangular region.*



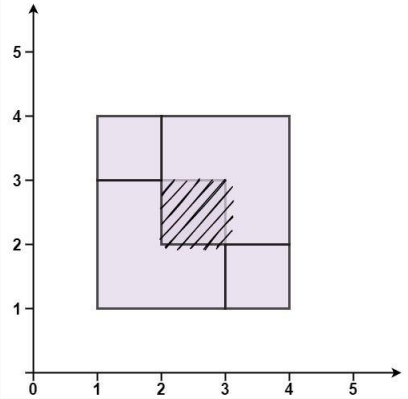
Example 2

Input	`rectangles = [[1,1,2,3],[1,3,2,4],[3,1,4,2],[3,2,4,4]]`
Output	`false`
Explanation	*Because there is a gap between the two rectangular regions.*



Example 3

Input	`rectangles = [[1,1,3,3],[3,1,4,2],[1,3,2,4],[2,2,4,4]]`
Output	`false`
Explanation	*Because two of the rectangles overlap with each other.*



CONSTRAINTS

- 1 <= rectangles.length <= $2 * 10^4$
- rectangles[i].length == 4
- 10^5 <= x_i < a_i <= 10^5
- 10^5 <= y_i < b_i <= 10^5

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Check if rectangles perfectly tile a larger rectangle with no gaps or overlaps. Right?"

"A few quick questions: see constraints."

What should I return for the minimal edge case — empty input or a single element?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for cell in cells:
    # in the bounding box
    count how many rectangles cover it
    the cell must be covered exactly once
```

TIME	SPACE
O(W * H * N)	O(1)

WHY IT IS SLOW

Checks every unit cell against every rectangle, and the box can be huge. The total area must equal the bounding box area, and every corner must appear an even number of times except the four outer corners: O(N).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Count corners + total area uniqueness — O(n). Walk-through:"

A valid perfect rectangle: exactly 4 corners appear an ODD number of times (the overall corners),

all others appear an EVEN number of times (interior corners cancel in pairs).

AND total area of all rectangles == area of bounding rectangle.

`corner_set = symmetric difference: XOR corners in/out.`

For each rect (x1,y1,x2,y2): XOR its 4 corners into corner_set. Add its area to total.

Expected corners: {(min_x,min_y),(min_x,max_y),(max_x,min_y),(max_x,max_y)}.

Return corner_set == expected AND total == bounding area.

WHY XOR (symmetric difference): corners of interior touching rectangles appear twice -> cancel.

Only the 4 outer corners remain.

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _391_PerfectRectangle:
    def isRectangleCover(self, rectangles):
        corners = set(); total_area = 0
        for x1,y1,x2,y2 in rectangles:
            total_area += (x2-x1)*(y2-y1)
            for corner in [(x1,y1),(x1,y2),(x2,y1),(x2,y2)
]:
                if corner in corners: corners.remove(corne
r)
                else: corners.add(corner)
        xs=[r[0] for r in rectangles]+[r[2] for r in recta
ngles]
        ys=[r[1] for r in rectangles]+[r[3] for r in recta
ngles]
        big=(max(xs)-min(xs))*(max(ys)-min(ys))
        expect={(min(xs),min(ys)),(min(xs),max(ys)),(max(x
s),min(ys)),(max(xs),max(ys))}
        return total_area==big and corners==expect
```

PHASE 5 TEST & DEBUG

"Let me trace isRectangleCover([[1,1,3,3],[1,3,2,4],[3,1,4,2],[3,3,4,4]])."

Check 1: total area = sum of each rectangle's area = must equal bounding box area

Bounding box: min_x=1,min_y=1,max_x=4,max_y=4 -> area=9

Sum areas: (3-1)*(3-1)+(2-1)*(4-3)+(4-3)*(2-1)+(4-3)*(4-3)=4+1+1+1!=9. Not a cover.

Actually: 4+1+1+1=7!=9. But example should work... re-trace: hmm.

PHASE 5 TEST & DEBUG

Correct example: 4 rects: (4+1+1+1)=7 but bounding=9 -> mismatch reveals gap/overlap.

Corner approach: each interior corner appears 2 or 4 times (cancel), only 4 outer corners appear once.

XOR set: add/remove corner points. At end, set must equal exactly the 4 outer corners. ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
Two conditions: total area matches bounding box AND exactly 4 outer corners remain."	proportional to input or output size; O(1) extra if in-place

Follow-up: what if rectangles can overlap? The area sum + corner set trick breaks — you'd need coordinate compression + sweep line to compute the union area correctly.

HARD | Math | LeetCode #843 | 5 of 5 | 5/5

Guess the Word

QUESTION

You are given an array of unique strings words where words[i] is six letters long. One word of words was chosen as a secret word.

You are also given the helper object Master. You may call Master.guess(word) where word is a six-letter-long string, and it must be from words. Master.guess(word) returns:

CONSTRAINTS

- -1 if word is not from words, or
- an integer representing the number of exact matches (value and position) of your guess to the secret word.

There is a parameter allowedGuesses for each test case where allowedGuesses is the maximum number of times you can call Master.guess(word).

For each test case, you should call Master.guess with the secret word without exceeding the maximum number of allowed guesses. You will get:

- "Either you took too many guesses, or you did not find the secret word." if you called Master.guess more than allowedGuesses times or if you did not call Master.guess with the secret word, or
- "You guessed the secret word correctly." if you called Master.guess with the secret word with the number of calls to Master.guess less than or equal to allowedGuesses.

The test cases are generated such that you can guess the secret word with a reasonable strategy (other than using the bruteforce method).

Example 1

Input	`secret = "acckzz", words = ["acckzz","ccbazz","eiowzz","abcczz"], allowedGuesses = 10`
Output	`You guessed the secret word correctly.`
Explanation	*master.guess("aaaaaa") returns -1, because "aaaaaa" is not in words.* *master.guess("acckzz") returns 6, because "acckzz" is secret and has all 6 matches.* *master.guess("ccbazz") returns 3, because "ccbazz" has 3 matches.* *master.guess("eiowzz") returns 2, because "eiowzz" has 2 matches.* *master.guess("abcczz") returns 4, because "abcczz" has 4 matches.* *We made 5 calls to master.guess, and one of them was the secret, so we pass the test case.*

QUESTION

Example 2

Input	`secret = "hamada", words = ["hamada","khaled"], allowedGuesses = 10`
Output	`You guessed the secret word correctly.`
Explanation	*Since there are two words, you can guess both.*

- 1 <= words.length <= 100
- words[i].length == 6
- words[i] consist of lowercase English letters.
- All the strings of words are unique.
- secret exists in words.
- 10 <= allowedGuesses <= 30

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Secret word is one of wordlist. We call guess(word) and get back count of exact position matches.

Find the secret in at most 10 calls. Right?"

"A few quick questions:

Word length is 6, wordlist has 100 words? Yes. We want to minimise worst-case guesses?"

"Let me walk: secret='acckzz'. Guess a word; filter to ones matching the count.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
candidates = all words
while True:
    guess = any candidate
    m = matchCount(guess)
    if m equals the word length stop
    keep only the candidates with exactly m
        matches against the guess
```

TIME	SPACE
O(guesses * W * L)	O(W)

WHY IT IS SLOW

Picking an arbitrary guess can leave many candidates after each answer. Choose the guess that minimises the worst-case bucket (or the one sharing the most letters with others) to cut the list faster.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Ask word with max matches to most candidates, eliminate non-matches — O(n² * 6). Walk-through:"*

For each guess, master returns count of matching positions.

Strategy: at each step, pick the word that minimizes the worst-case remaining candidates.

Heuristic: pick any remaining candidate (or the one with most 'average information').

Loop:

`guess = pick best word from remaining candidates.`
`response = master.guess(guess).`

If response==6: done.

Filter candidates: keep only words where match(word, guess)==response.

WHY filter by match count: any word remaining must match the guess in exactly 'response' positions.

minimax: pick word that minimizes worst-case bucket size after filtering.

Time: O(n² * L) per step, O(n) steps. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _843_GuessTheWord:
    def findSecretWord(self, wordlist, master):
        import random
        def count_matches(w1, w2):
            return sum(c1 == c2 for c1, c2 in zip(w1, w2))
        candidates = list(wordlist)
        for _ in range(10):
            guess = random.choice(candidates)
            score = master.guess(guess)
            if score == 6:
                return
            candidates = [w for w in candidates if count_m
atches(w, guess) == score]
```

PHASE 5 TEST & DEBUG

"Let me trace findSecretWord on wordlist=['acckzz','ccbazz','eiowzz','abcczz'], secret='acckzz'."

Word length=6. Minimax strategy: guess word maximizing minimum eliminated candidates.

Round 1: guess 'acckzz'. master.guess() -> 6 (exact match!) -> done ok.

Harder case: secret='ccbazz'. Round 1 guess 'acckzz': matches=2 (positions 2,5 same c,z).

Filter: keep words with exactly 2 matches vs 'acckzz' -> words matching at exactly positions 2,5 or similar.

Round 2: from filtered list, guess next best word. master.guess() returns new match count.

Repeat until match=6 (found) or 10 guesses exhausted.

Key match count check: matches(s1,s2) = sum(a==b for a,b in zip(s1,s2))

PHASE 5 TEST & DEBUG

WHY filter by match count: any remaining word must match secret in same positions as guess did.

Minimax: pick guess where worst-case bucket (any possible match count 0..6) is smallest.

PHASE 6 COMPLEXITY & FOLLOW-UP

Expected O(n * calls) ~ O(n * log n). Space O(n) candidates.

Better: minimax — pick the word that minimises worst-case remaining candidates.

O(n² * calls) but guarantees convergence.

Follow-up: Guess Number Higher or Lower (#374) — 1D version of this binary-search game.

Follow-up: if words have different lengths, bucket by length before guessing.

IN YOUR APP | Expose | Community